Steamy Vampires and a Dirty Dataset

Information Visualization, summer 2024

Sofie Kirk Nielsen, Hana Spáčilová, Jacob Laust Høy (Group 1)
Department of Computer Science, Aarhus University

August 16, 2024

Abstract

We present an interactive visualization solution for the task of exploring a big *Goodreads* dataset and of identifying sub-genres in the fantasy genre using a colored nodes-links graph. The final solution does not entirely solve the posed task, but limitations are discussed through a description of our design process and the data being used, and we discuss possible future improvements that might rectify these inadequacies.

1 Motivation

In the beginning there was a big, dirty dataset with many fantasy books containing information entered by users. One piece of information is the description. Early on, we wanted to build a visualization that could aid users in finding their next favorite book according to their tastes and preferences. To this end, we wrote a small program which gave us a list of all books with certain keywords in their description.

Because the dataset also includes many fan-fiction books, we got many amusing and interesting results. For example, out of *216.345* books, *244* of them included the words "steamy" and "vampire". We wanted to create a tool for users that would enable them to find more books similar to their preferences through an exploration of the dataset. Furthermore, our goal was to make it more user friendly compared to what is offered on *Goodreads*. If we look at the output (for the code, see Appendix B) with results showing titles and descriptions of the books with the two above-mentioned keywords, we get a long text (see Appendix C), which is not visually appealing. Therefore, we came to the conclusion that showing the results in a visual way would be much more useful.

Another possible reason for users to use our visualization is if a they have a crazy idea for a book to write about some topic, then they can not only check to see whether a similar book has already been written on said topic, but also gauge the popularity of such an idea.

During the data analysis and preliminary research stage we came upon a network graph visualizing the character interactions in the novel *Les Misérables* [7, p.6-7] and were inspired to test the usage of a similar graph approach to visualize our book data. The node graph would provide a quick and simple overview of a user-selected subset of a large dataset which would provide the user a way to see what kind of books cluster together (which suggests that these books are similar) and what kinds of books are on there own (suggesting that these solitary books are unique).

The similar books section of *Goodreads* provides a way for the user to get suggestions for future reads, but our graph will make it possible for the potential reader to discover future reads to a greater extent by looking at the proximate nodes (since we can assume that nodes that are indirectly connected through one other node but not with each other are also similar to some degree). A color scale for the nodes will provide a way to see whether certain types of books are rated similarly, and what sort of books are rated highly (or poorly).

In concrete terms, we have two main objectives with our interactive visualization:

- Let users find new books that might suit their interests.
- Identify sub-genres by finding sub-clusters in the network graph.

In more abstract terms, the main tasks of the visualization are:

- To explore the dataset.
- To provide an overview with the possibility to show details on demand.

2 Description of the visualization problem

Having stated the motivation for making the visualization in the previous section, we now succinctly describe the visualization problem in this section which is then followed by a discussion of our solution to this problem in section 3.

In broad terms, the problem is thus to turn a very large and dirty dataset into a beautiful and intuitive interactive visualization in the form of a network graph that provides an overview, zooming/panning and the possibility of showing details on demand, thus following Shneiderman's mantra. Turning the large, dirty, and unwieldy dataset into something that a user can easily take in, is a challenge in several ways. In practice, this requires processing and formatting a very large JSON-file into something that can be fed into a visualization tool. It then requires us to provide tweaks and additions that aid user-friendliness.

2.1 Data

Our original dataset *goodreads_books_fantasy_paranormal.json* includes 258,585 books[8]. The dataset was created by a user-input, where the users could add values to the list of attributes about the books, e.g. *title*, *similar_books*, *rating*, *publication_year*, etc. (for the full list of attributes, see Appendix A). Since all data were entered by different users and not checked for the right format, many of the items in our dataset have missing values as well as noticeable amount of misspelled values.

As an example, we can mention the values for the attribute *publication_year*[1]. There are *216.345* items in the reduced dataset. Most of the items have relevant values, *158.566* items with valid years between *1900* and *2024*. However, *78* items have year lesser than *1900* and *33* items more than *2024*, while *57.688* item have no recorded value.

3 Design process and visualization solution

In this chapter we discuss how we realized the visualization so that it would fulfil our stated purpose and our thought process along the way. We describe this from the perspective of the visualization pipeline (see Figure 1).

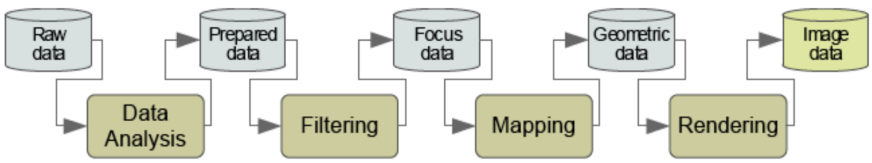

Figure 1: Visualization Pipeline, taken from [2].

In broad terms, the problem is solved through a user-defined filtering of the dataset that then results in a node graph that is easy to navigate and explore. This will also result in clustering of similar books which we can use to create some kinds of subsets of the different types of books in the dataset. The clusters are then to be understood as forming sub-genres within the fantasy genre. This will also enable a user to find books based on some criteria (keywords or rating) and get to see other books with similar features.

We used theoretical knowledge about data visualization from *Interactive Visual Data Analysis*[7] and from lectures and exercises while attending the summer course *Information Visualization*[1]. After choosing our dataset and deciding on the goal of our visualization, we ran into many unfortunate problems with our dataset (see below). Therefore, we had to spend a lot of time on pre-processing and

[1]The numbers are taken from the version of dataset with books in English and with books without value for language.

on creating new raw datasets that could be used for further analysis. We alternated many times between changing the raw data and analysing it. Even though in practice we worked on pre-processing and analysis concurrently, we present both steps in the following subsections separately. In the subsection about data pre-processing, we focus on the problems we had with the dataset and in the subsection about data analysis, we focus on concrete parts of the dataset that we used and on how we used it.

Because we followed the visualization design pipeline [2], the following subsections present the subsequent steps in the pipeline, i.e. we will talk about filtering, mapping, rendering, and interaction.

3.1 Data Pre-processing and raw data

As mentioned above, our dataset is far from clean and consistent. We started by focusing on the missing values. First, we considered data amputation, i.e. we removed all the items that were missing attributes that we wanted to focus on, e.g. *title*, *description*, *publication_year*, *average_rating*, or *similar_books*. This approach left us with a cleaner subset. Unfortunately, this new dataset only had *2692* items. This would not be a problem on its own, if we wanted to use the data differently. However, an important part of our interaction with data was the usage of the attribute *similar_books* which is a list with book-id's of other books with similar features. Getting the reduced dataset meant deleting many of the books that the similar book id's pointed to. This meant that the lists pointed to non-existing books. Similar problems arose for books with the same keywords. These we searched from the description written in the attribute *description*. E.g. because most of the items were removed, the search for some keyword outputted only one book, and this made further clustering impossible.

We have dismissed an imputation approach because with the exception of the *publication_year* and some other attributes, the values were entered based on subjective judgement and we did not manage to come up with any function which could add the missing values. This is because there is no apparent correlation between the attributes.

We were therefore forced to accept the disadvantages of using an unbalanced dataset and ended up using most of the items. This meant that some items would be missing in some parts of the visualizations if we were to run into a missing value of an attribute in focus.

We filtered out some of the columns and left only the rows written either in English or with a missing language value, because the group of items without information on language was quite big (70.664 items). Because of the reasons against amputation stated above, we decided to keep this group in our dataset.

The original dataset includes the object *popular_shelves* which is a long list of different attributes and values by which users sort the books into sub-genres or types of books. Originally, we thought that this could be useful for clustering purposes. However, the objects were long and without any standardized format. Furthermore, different values had different amounts of user votes, and we eventually decided to focus on making the clusters based on the *similar_books* attribute and on using keywords search in the *description* text.

When we were working on pre-processing and data analysis, we confirmed that a node-link network graph would be useful. But in order to make it work we had to pre-process our data again because we needed to transform it into a *.json* file with two objects: one with nodes where the attributes and values from the pre-processed dataset were stored, and one with links in the form of tuples (source - target). Here the source is a book id and the target is the id of one of the books in its similar books list. This was a necessary step to make a network diagram.

3.2 Data Analysis and Prepared data

For our purposes, the most useful attributes were the *similar_books* and *description* attributes. The *similar_books* together with *book_ids* were necessary for creating the tuples for the source-targets of the graph. Here we use information already given in the dataset to create the clusters of similar books, while subsets of data-items created based on keywords from *description* was meant to be fully dependent on choices made by users of the visualization. Combining both ways of finding clusters should make us able to get better results than if we were to use only one of them.

We also considered using the attribute list *authors*, however, the names of the author or authors were not specified by their actual name, but by a numerical value *author_id*, which had to be searched in another database on *GoodReads*' website. Using the authors would have been a useful tool to create clusters even more efficiently, but we did not want to use another source of data, so this list attribute was quickly dismissed. Since we could not identify the authors for each book, we included instead

information about the publisher. In this way, if a user wanted to google the book and find further information he/she would be able to. And of course, we used the names of the books from the attribute *title_without_series*.

Another attribute in consideration was *average_rating* which could make the dataset easier for user filtering, but the analysis of the rating did not lead to any useful tools. However, we included the rating into the attributes of our nodes.

3.3 Filtering and Focus data

As mentioned above, we focused mostly on attributes *similar_books* and *description*, which we used for finding the clusters of books with similar features. We wanted users to choose a keyword[2]. The keyword can either be chosen by clicking on three available options or by entering another keyword into the search field. Then the network graph is created based on the keyword and nodes which have the keyword in the description are displayed together with links to the similar books. Users can further interact with the graph by entering a keyword to highlight nodes that also include this keyword.

After showing the overview and filtered graph, users also have the option to get details on demand by hovering over nodes. This will display name, publisher and rating of the book. Clicking on a node will show a text-box with a full description of the book.

3.4 Mapping and Geometric data

The books are represented as nodes in the form of circle. The Viridis color scale is used for the mapping of book ratings (0.0 - 0.5) onto color saturation. The monotonically increasing luminance of the Viridis color scale allows for optimal mapping of continuous data using a magnitude channel. Since the majority of books were rated 3.0 or higher, we decided to shrink the color scale so that it only covered the range 3.0 - 5.0 (and everything below 3.0 is colored as if it were 3.0 (in yellow)). Furthermore, the scale has been inverted, as it was found (based on a small in-person survey) that people expected the lighter colors to be associated with lower ratings and the darker colors to be associated with higher ratings.

Orange has been chosen as the highlighting color. It has the disadvantage that it might not be colorblind safe against green (which is a part of the Viridis color scale), but has the advantage that it contrasts with azure blue (which is somewhere in the middle of the Viridis scale). To accommodate colorblind users we have also made the nodes significantly bigger when highlighted. Size is ranked low on MacKinley's visual variables sorting for categorical data [3, p.125], but since we only have two categories ("contains keyword" and "does not contain keyword"), we judge it to still be effective. We could have used shape as an identity channel, but decided against it since we already have two ways of distinguishing nodes that either contain or do not contain the keyword.

An overall better solution (that we unfortunately did not have time to implement) would be to use a monotonically increasing color scale in a single color hue (e.g. blue) for the non-highlighted nodes and a monotonically increasing color scale in a different color hue that contrasts with blue (i.e. red) for the highlighted ones. In this way, the user would easily be able to identify the highlighted nodes (since red is a signal color); it would be colorblind safe (because blue-red is not a problematic combination); and the user would be able to easily see from the saturation on both the highlighted and non-highlighted nodes how highly the book is rated.

3.5 Rendering and Image data

In the rendering stage, we tested where to place the elements of our visualization and how to display them. One of our main tasks was to find clusters, and the overview graph already works well to show these clusters. But if a user wants to see details, the nodes and links are often placed on top of or are too close to each other. To resolve this, we implemented a function tied to the button *Enable Collision Detection* which forces nodes apart, so that it becomes easier to read and interact with them.

User-friendliness was also the reason behind the implementation of *Pause Simulation*. Without this implementation users must wait until the whole simulation is completed and interacting with the nodes becomes difficult. Pressing the button enables them to start interacting with the graph immediately, e.g. if they see something interesting.

[2]We did not manage to implement the choice of more than one keyword, but the functionality of the second keyword is implemented as a *highlight* filtering option. The further development would include implementation of both more keywords and highlights. For an explanation see below in Section 5.

3.6 Interaction with data and user testing

Users can use a slider or the mouse-wheel to zoom in and out. This enables them to see the overview of the data in the filtered graph and identify the clusters or to zoom-in to see details about the nodes.

A possible future improvement would be to aid visual information seeking by implementing either overview and detail, having two separate views of the total graph and the details being zoomed in on; or by implementing focus and context, having the zoomed in area embedded into the larger total graph. We have tested our implementation extensively with different search and highlight words to make sure that our implementation is user friendly.

4 Example use case

A user might start by searching the dataset for the keyword 'vampire'. This will generate a node-links graph of 2000 vampire books providing an overview to the user that shows the user whether books containing this keyword tend to be connected and to what degree they are, judging from the size of clusters. The user can then choose to zoom in on a particular cluster.

Filtering by a second keyword (e.g. 'werewolf') through the 'highlighting' function will grant the user an instant overview of not just the fraction of vampire books that also contains werewolves, but will also allow the user to see whether particular keywords tend to facilitate more clustering which could be taken as a sign that books containing vampires *and* werewolves are also seen as more similar, or whether the presence of the two keywords does not signify any connection between books. Having explored the graph through zooming, traversing and filtering, the user might then call up details on demand related to the books rating, publisher and description. In this way, the user's interaction with the visualization neatly follows Shneiderman's visual information seeking mantra [6] of "overview first, zoom and filter, then details-on-demand".

5 Limitations and Further Improvements

Our implementation suffers from a number of limitations. As was mentioned in the beginning, the dataset was "dirty" which led to us spending too much time on pre-processing. Originally, we focused on implementation with the library *Vega-Altair: Declarative Visualization* for interactive visualization[9] and the package *NetworkX* for graph creation[4]. However, during our working process we switched to the JavaScript library *D3*[5]. This unfortunate change meant that we had to use extra time becoming familiar with the new library as well as spending time switching from Python to the holy trinity of web-development.

Difficulties, partially caused by the change of library and language meant that we did not manage to fully implement what we originally intended, but instead managed to present a visualization using a smaller dataset which still allows us to show what we had envisaged in the beginning.

The original goal of identifying sub-genres within the dataset was somewhat hampered by the fact that the dataset only contains short descriptions of each book. This was found to not be adequate for the purpose of actually naming and labeling the clusters that appear on the node graph. A dataset that contains a well formatted list of user-chosen keywords for each book would make this more feasible.

Although the dataset has been reduced from its original size by removing books in non-English languages, books without a description etc., after those removals the dataset has been arbitrarily amputated at 25,000 entries based only on the order of their position in the dataset (and this position has no inherent meaning). Using the entire dataset (or at least a smaller version that has been reduced based on sensible consideration) would be better, but this was not done due to technical difficulties.

The use of color could be improved as mentioned in section 3.4.

A nice improvement would also be the addition of the possibility of the user to search for a particular book of interest so that the user could see whether this book clustered together with other books (that the user might be interested in reading in the future).

6 Group responsibilities

The report writing and sharing thoughts and problems, as well as discussing issues and finding solution were made jointly by all three group members.

All group members were originally working separately, because we wanted to explore, learn and find options separately. Then, we created a Github repository: `https://github.com/hankaspac/info_vis`, where we shared our code. First, we worked on separate branches and then we moved our work to *working_branch*, where we finished our implementation and merged into the main branch. The branch *original_working_branch* (Jacob was the main contributor) includes the first stage of the graph implementation, which the *working_branch* (Sofie was the main contributor and then Jacob and Hana joined the collaboration on this branch) is based on and we worked jointly in the last stages of our implementation. The last branch *not_finished_branch* (Hana was the main contributor) is an unfinished attempt for another approach by using *Altair* library, but we did not get far with that one.

7 Conclusion

We set out to solve the problem of visualizing a very large dataset in a way that would support two main tasks. In concrete terms, we wanted to 1) identify sub-genres within the fantasy genre, and 2) we wanted to aid users in finding new books. Abstracting these tasks, we wished to aided in the exploration of and to provide an intuitive overview of a very large dataset. We decided on a node graph as a viable solution to this problem.

Judging the final visualization solution, in our own humble opinion, the visualization is first of all expressive in that it manages to show the underlying data that a user might be interest in without showing anything more than is already there. The expressiveness is supported by the choice of graph which offers users an overview that is not needlessly complex. The data being shown is faithfully expressed and does not imply connections that are not present in the underlying dataset. I.e. the visualization does not show any additional and/or misleading information and it would be hard to misinterpret any conclusions made based on the graph.

Second of all, the visualization is effective in that it shows the connections between similar books in an understandable way. It does this by using the relative positions (a powerful perceptual task on the list of mapping criteria by MacKinlay[3]) of the nodes to show connections among books. The visualization also leverages color in two ways; an intuitive color grading is used to indicate the rating of individual books, and a contrasting color (as well as size) is used to indicate highlighted books chosen by the user. The effectiveness could however be improved by using an even better color scheme, as discussed in section 3.4.

Finally, if we were to comment on the efficiency of the visualization, we might note the fact that the chosen visualization (node-links graph) is not the easiest graph to make. Efficiency is based on the criteria that the visualization is actually worth making from the creator's standpoint and worth using from the user's standpoint.

Comparing our solution to the simple outputted list with titles and description, our visualisation obviously has clear benefits over no visualization at all, because the long text does not include any way for us to see clusters and relations between single items. Although simpler visualizations exist, we do not find that these alternative visualizations would provide[3] the same support for the originally stated goals of finding clusters and providing users a means to find new books, i.e. the tasks of providing overview and a means of exploring the dataset. From the user's point of view, the visualization is fairly user-friendly, but actually gauging user-friendliness would require further testing and possibly user focus-grouping.

References

[1] AU. Course catalogue - information visualization, summer 2024. https://kursuskatalog.au.dk/en/course/125549/Information-Visualization. Accessed: 15/08/2024.

[2] Information Visualization community platform. Visualization pipeline. https://infovis-wiki.net/wiki/Visualization$_{P}ipeline.Accessed$: 15/08/2024.

[3] Jock Mackinlay. Automating the design of graphical presentations of relational information. *ACM Trans. Graph.*, 5:110–141, 04 1986.

[4] NetworkX. Networkx - network analysis in python. https://networkx.org/. Accessed: 15/08/2024.

[5] Observable. The javascript library for bespoke data visualization. https://d3js.org/. Accessed: 15/08/2024.

[6] B. Shneiderman. The eyes have it: a task by data type taxonomy for information visualizations. In *Proceedings 1996 IEEE Symposium on Visual Languages*, pages 336–343, 1996.

[7] C. Tominski and H. Schumann. *Interactive Visual Data Analysis*. ISSN. CRC Press, 2020.

[8] GoodReads' users. Fantasy paranormal (258,585 books, 55,397,550 interactions, 3,424,641 detailed reviews). https://mengtingwan.github.io/data/goodreads?fbclid=IwY2xjawEpbTRleHRuA2FlbQIxMAABHbeV-aNr6PF2uM4QovGofDN$_{z}zeKAqa4YUzrYP3EWNiQEWYkBvAhVNlPnA_{a}em_{l}127UCqAyFhp17wS5JaqtAdatasets.Accessed$: 04/07/2024.

[9] Vega-Altair. Vega-altair: Declarative visualization in python. https://altair-viz.github.io/. Accessed: 15/08/2024.

A List of attributes for the original dataset

```
isbn                    object
text_reviews_count       int64
series                  object
country_code            object
language_code           object
popular_shelves         object
asin                    object
is_ebook                object
average_rating         float64
kindle_asin             object
similar_books           object
description             object
format                  object
link                    object
authors                 object
publisher               object
num_pages               object
publication_day         object
isbn13                  object
publication_month       object
edition_information     object
publication_year        object
url                     object
image_url               object
book_id                  int64
ratings_count            int64
work_id                  int64
title                   object
title_without_series    object
```

Figure 2: List of attributes and their types in the original dataset

B Code for keywords search

```
import altair as alt
import pandas as pd
import vegafusion as vf
import json
import numpy as np

alt.data_transformers.enable('json')

with open('graph_nodes.json') as file:
    books = json.loads(file.read())

my_chart = pd.json_normalize(books)

keywords_description = ["steamy", "vampire"]

def books_with_keywords(words):
    count = 0
    text_file = []
    for i in range(len(my_chart['description'])):
        if words[0] in my_chart['description'][i] and words[1] in my_chart['description'
            count += 1
            if my_chart['title_without_series'][i] == '':
                text_file.append("The book does not have a title.")
            else:
                text_file.append(my_chart['title_without_series'][i])
                text_file.append("\n")
            text_file.append(my_chart['description'][i])
            text_file.append("\n \n \n")

        else:
            continue
    print(count)
    return text_file

text_file = books_with_keywords(keywords_description)

with open("file_words.txt", "w") as output:
     output.writelines(text_file)
```

C Outputted list

For the excerpt from the 122 pages of the outputted list, see external appendix called "Appendix C: outputted list: first 10 pages".